

Adapter

Refactoring en el mundo real — Migración de proveedor con APIs incompatibles

Parámetro	Valor
Asignatura	Metodología de Sistemas II
Modalidad	Trabajo Práctico Individual
Tiempo estimado	2 a 3 horas
Lenguaje	Java, Python, C# o TypeScript
Restricción	Sin librerías externas

¿De qué trata este trabajo?

Esto no es un ejercicio teórico sobre definiciones. Es una simulación de lo que pasa todos los días en proyectos con dependencias de terceros: una librería deja de mantenerse y aparece un reemplazo con una API completamente distinta. Reescribir el sistema entero no es opción. Tu trabajo es resolver la incompatibilidad sin tocar lo que ya funciona.

Si entendés bien este patrón, tenés la lógica base para abordar cualquier patrón estructural de la familia GoF por tu cuenta.

Restricción global

No uses librerías externas. Todo se resuelve con el lenguaje base que elijas. El objetivo es demostrar que entendés el patrón, no que encuentraste una librería que lo implementa.

¿Qué está pasando en el proyecto?

El sistema usaba una librería de geolocalización que quedó sin mantenimiento. Se migró a un proveedor nuevo con una API completamente distinta. Todo el sistema —40 archivos— ya está escrito contra la vieja. No podés reescribir todo. Tampoco podés modificar el nuevo proveedor.

Las dos APIs que no se pueden tocar

```
// Lo que usa todo el sistema – no se puede cambiar
CLASE OldGeoService:
    MÉTODO get_location(ip) -> diccionario
        retorna: { lat: float, lng: float, city: string, country: string }

// Nuevo proveedor – no se puede modificar
```

```
CLASE NewGeoProvider:
  MÉTODO locate(ip) -> Objeto
    retorna objeto con estructura completamente distinta:
      .coordinates.latitude
      .coordinates.longitude
      .address.locality
      .address.nation

// Así está en los 40 archivos del sistema:
geo = NUEVO OldGeoService()
data = geo.get_location("200.45.123.10")
IMPRIMIR data["city"], data["lat"]
```

El problema de fondo

Tenés una pieza nueva con forma de cuadrado tratando de entrar en un agujero redondo. Reescribir los 40 archivos es inviable. Modificar el proveedor nuevo, tampoco es opción. Necesitás un traductor que, por fuera, parezca la API vieja, y por dentro use la nueva.

¿Qué hace el patrón Adapter?

Crea una clase intermediaria que, por fuera, expone exactamente la misma interfaz que el sistema espera (OldGeoService), pero internamente delega el trabajo al nuevo proveedor y traduce la respuesta. El sistema piensa que sigue hablando con el proveedor viejo.

Anclaje teórico

Adapter (también llamado Adaptador, Envoltorio, Wrapper) es un patrón de diseño estructural que permite la colaboración entre objetos con interfaces incompatibles. Actúa como un puente entre dos interfaces que, de otro modo, no podrían comunicarse entre sí.

La teoría distingue dos variantes de implementación:

- **Adaptador de Objetos (composición):** El adaptador implementa la interfaz que el cliente espera y envuelve al objeto incompatible mediante un campo. Funciona en cualquier lenguaje OOP.
- **Clase Adaptadora (herencia múltiple):** La clase adaptadora hereda interfaces de ambos objetos al mismo tiempo. Sólo viable en lenguajes como C++.

El ejemplo canónico del PDF (SquarePegAdapter extiende RoundPeg y contiene un campo SquarePeg) usa la variante de Adaptador de Objetos. Este ejercicio replica exactamente esa estructura: GeoServiceAdapter extiende OldGeoService (interfaz que el cliente espera) y compone un NewGeoProvider (el adaptee incompatible).

Roles según la teoría:

- **Cliente:** Los 40 archivos del sistema — contienen la lógica de negocio existente.
- **Interfaz con el cliente:** OldGeoService — describe el protocolo que el cliente sabe usar.
- **Servicio (adaptee):** NewGeoProvider — clase de tercero con interfaz incompatible.

- **Adaptador:** GeoServiceAdapter — implementa la interfaz del cliente y envuelve el servicio.

Diagrama UML

(Incluí en tu entrega un diagrama UML que muestre las cuatro piezas y la dirección de las dependencias.)

Pseudocódigo de la solución

La única clase nueva: el Adapter

```
CLASE GeoServiceAdapter EXTIENDE OldGeoService:
```

```
CONSTRUCTOR:
```

```
    this.provider = NUEVO NewGeoProvider()
```

```
MÉTODO get_location(ip):          // misma firma que el sistema espera
    resultado = this.provider.locate(ip)  // llama al nuevo proveedor
```

```
    // traduce la respuesta al formato viejo
```

```
    RETORNAR {
```

```
        lat      : resultado.coordinates.latitude,
```

```
        lng      : resultado.coordinates.longitude,
```

```
        city     : resultado.address.locality,
```

```
        country  : resultado.address.nation
```

```
    }
```

Cómo se usa — un único cambio en toda la app

```
// ANTES – en un archivo de configuración o fábrica:
```

```
geo = NUEVO OldGeoService()
```

```
// DESPUÉS – único cambio:
```

```
geo = NUEVO GeoServiceAdapter()
```

```
// Los 40 archivos siguen exactamente igual – no saben que cambió el proveedor:
```

```
data = geo.get_location("200.45.123.10")
```

```
IMPRIMIR data["city"], data["lat"]  // funciona igual
```

Cómo saber si lo hiciste bien

Si podés cambiar NewGeoProvider por un hipotético FutureGeoProvider modificando solo el Adapter, y sin tocar ningún otro archivo del sistema, el patrón está aplicado correctamente.

Tu misión

- Crear el GeoServiceAdapter como único archivo nuevo.

- No tocar OldGeoService, NewGeoProvider, ni ninguno de los 40 archivos del sistema.
- Demostrar que el cambio de proveedor es transparente para el código cliente.
- Responder en un comentario: ¿qué tendría que cambiar si llega un tercer proveedor mañana?

Restricciones

Solo se permite un archivo nuevo: el Adapter. La firma `get_location(ip)` debe mantenerse idéntica.

¿Qué espero ver en tu entrega?

Qué evalúo	Lo que estoy mirando
Diagrama UML	Cliente, OldGeoService, NewGeoProvider, GeoServiceAdapter — y las flechas correctas entre ellos.
Antes / Después	Cómo era una llamada antes del adapter y cómo queda después. Sin el contraste no sé si entendiste qué estaba mal.
Justificación	Máximo 10 líneas explicando por qué Adapter y no otra solución (refactorización masiva, fachada, etc.).
Transparencia	Que el código cliente realmente no haya cambiado. Es lo que valida que aplicaste el patrón.
Pregunta del 3er proveedor	El comentario sobre extensibilidad — quiero ver si pensaste el problema más allá del caso puntual.
Restricciones	Todas respetadas. Librería externa = ejercicio no cuenta, aunque el código sea brillante.

Última cosa

Si tu solución no encaja del todo con el patrón "canónico" del libro, explicá por qué la elegiste así. Eso me dice más sobre cómo pensás que si copiaste la implementación de un tutorial.